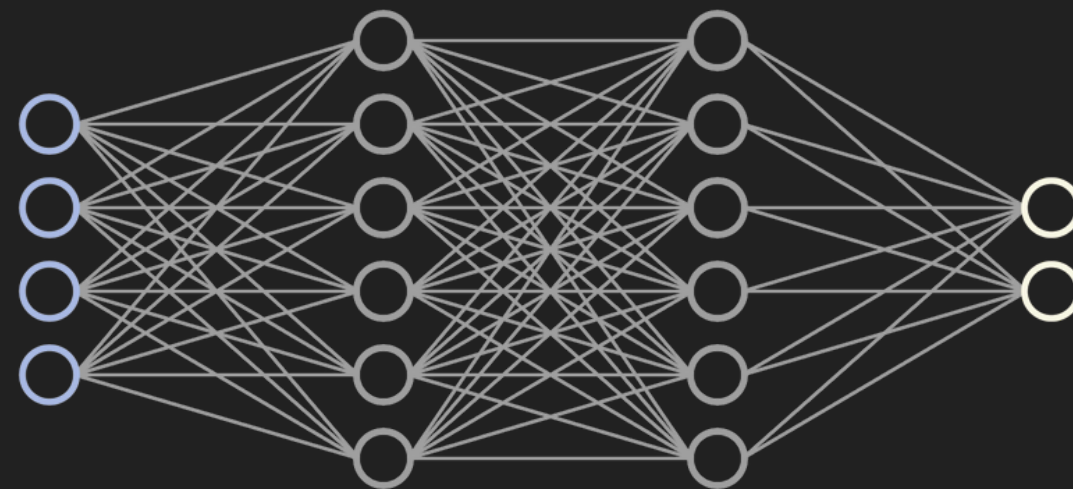




Google Developer Group
Universiti Teknologi Malaysia

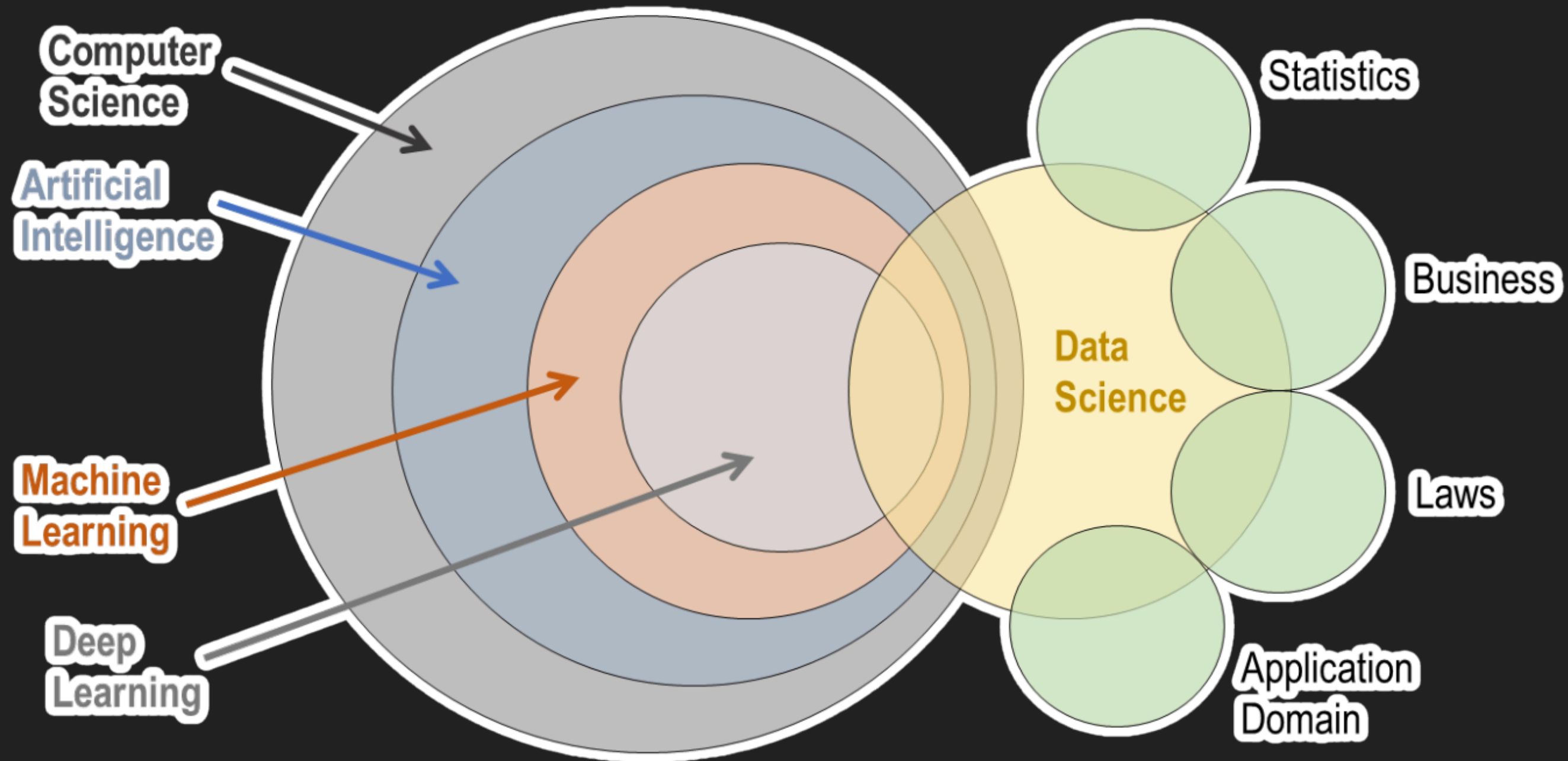


BASICS OF NEURAL NETWORKS

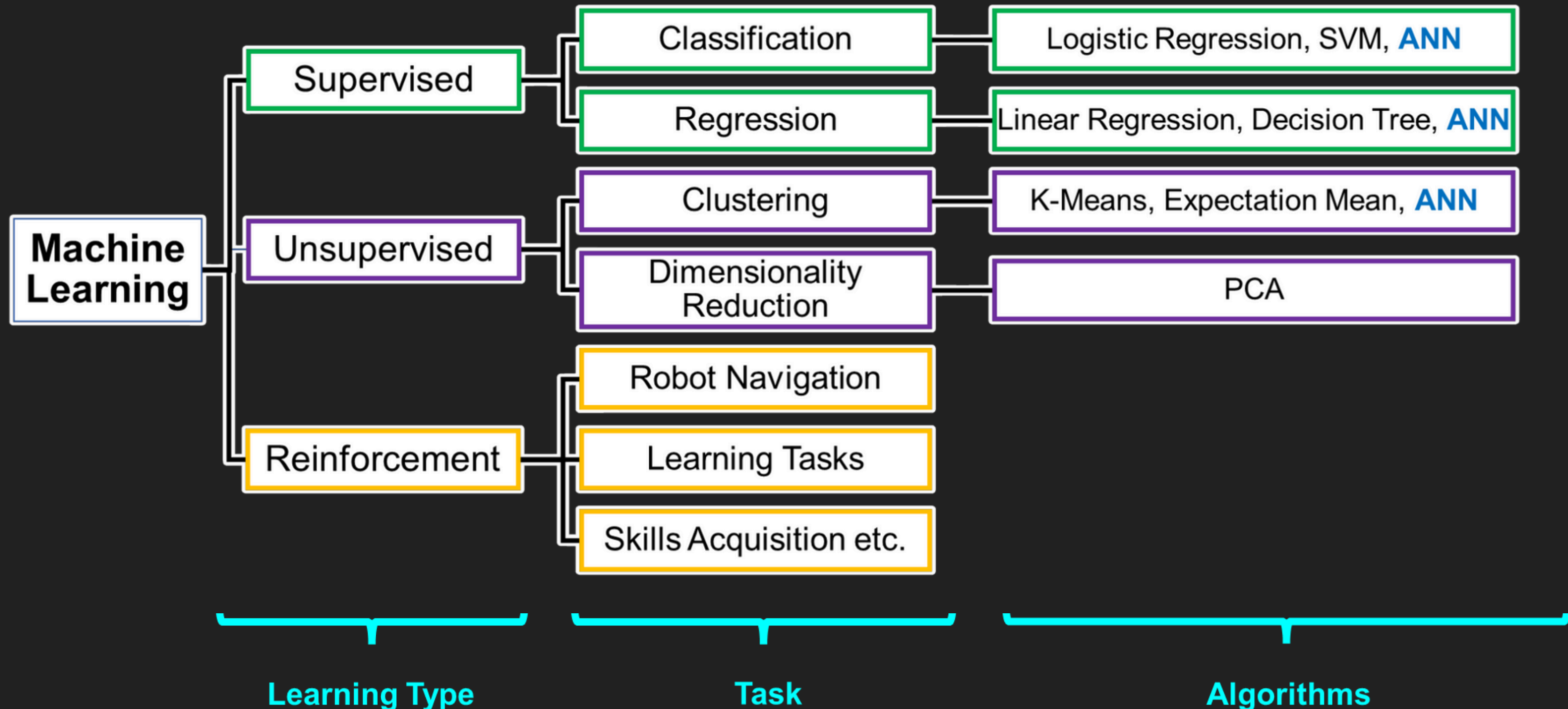


PRESENTED BY
ANAS ABURAYA

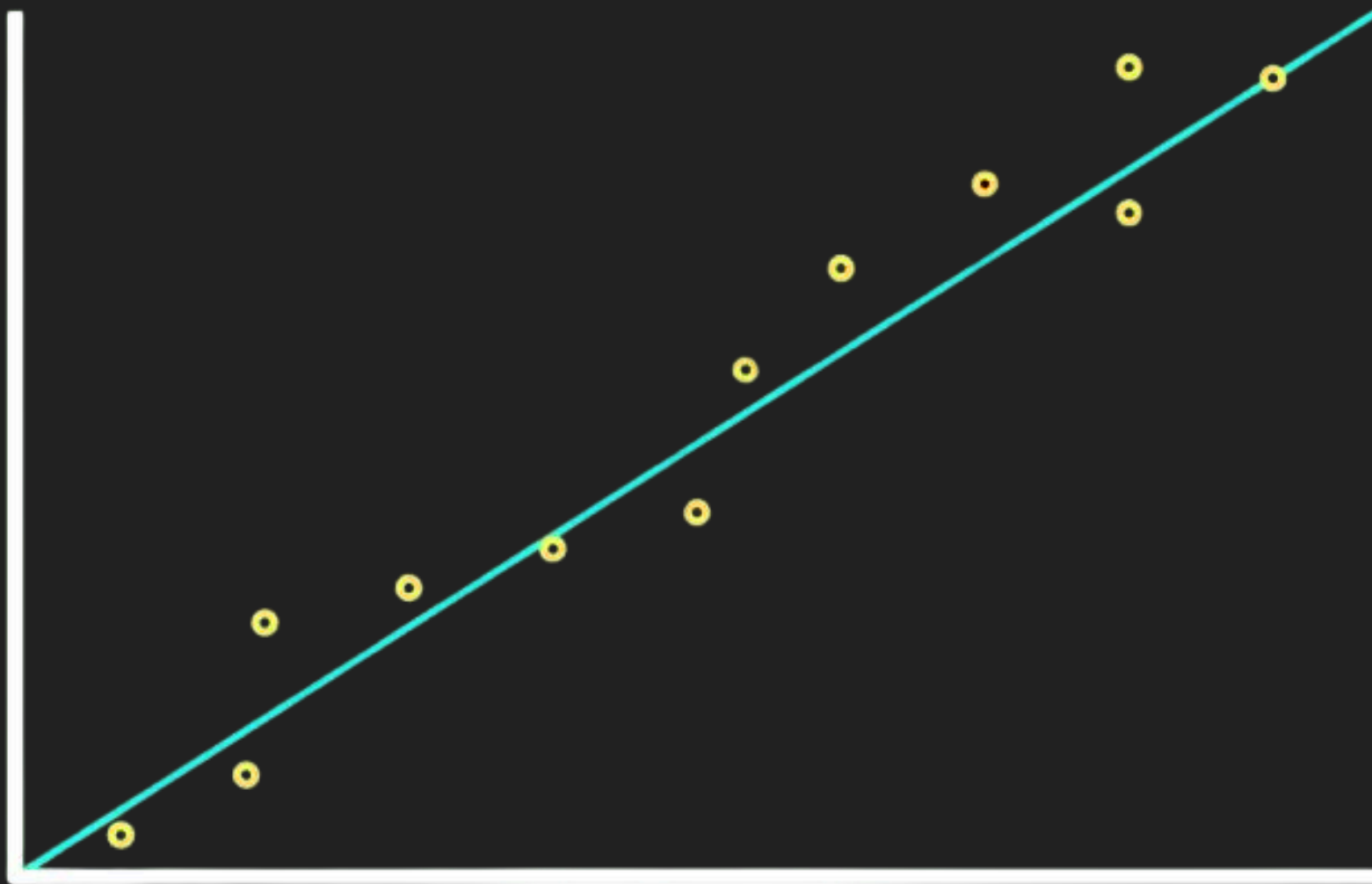
TERMINOLOGY



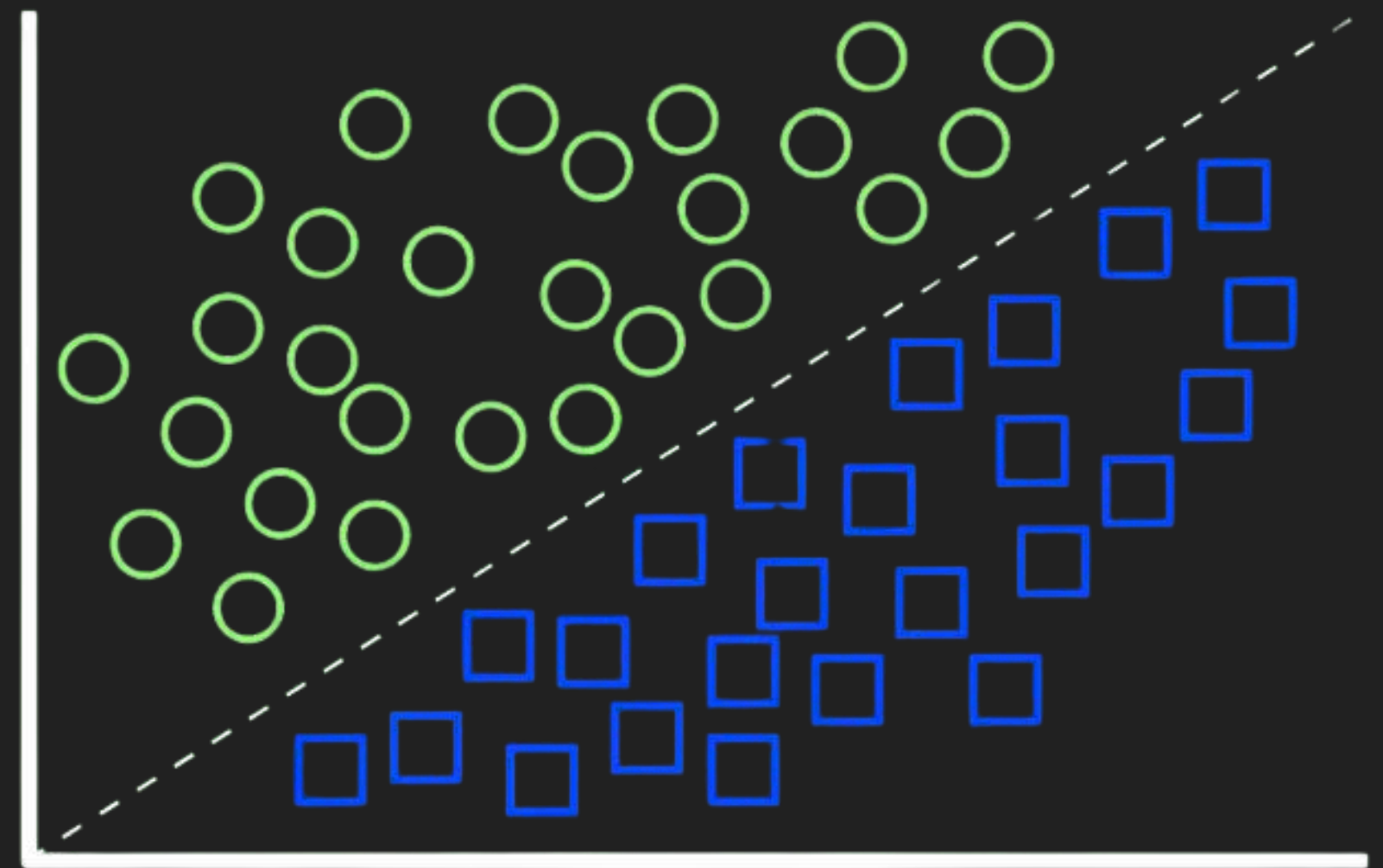
TYPES OF MACHINE LEARNING



SUPERVISED LEARNING TASKS



Regression

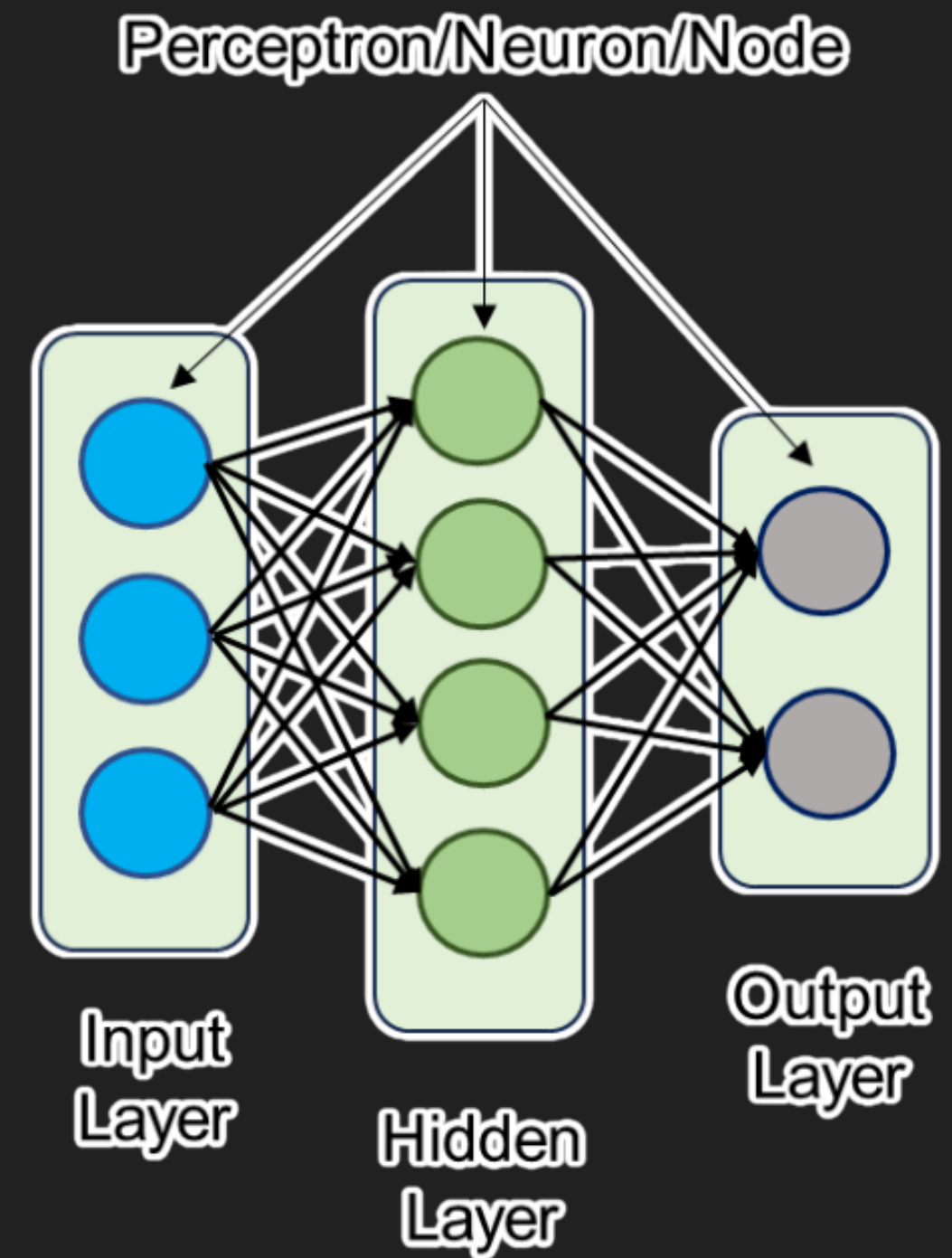


Classification

COMPONENTS OF SIMPLE ANN

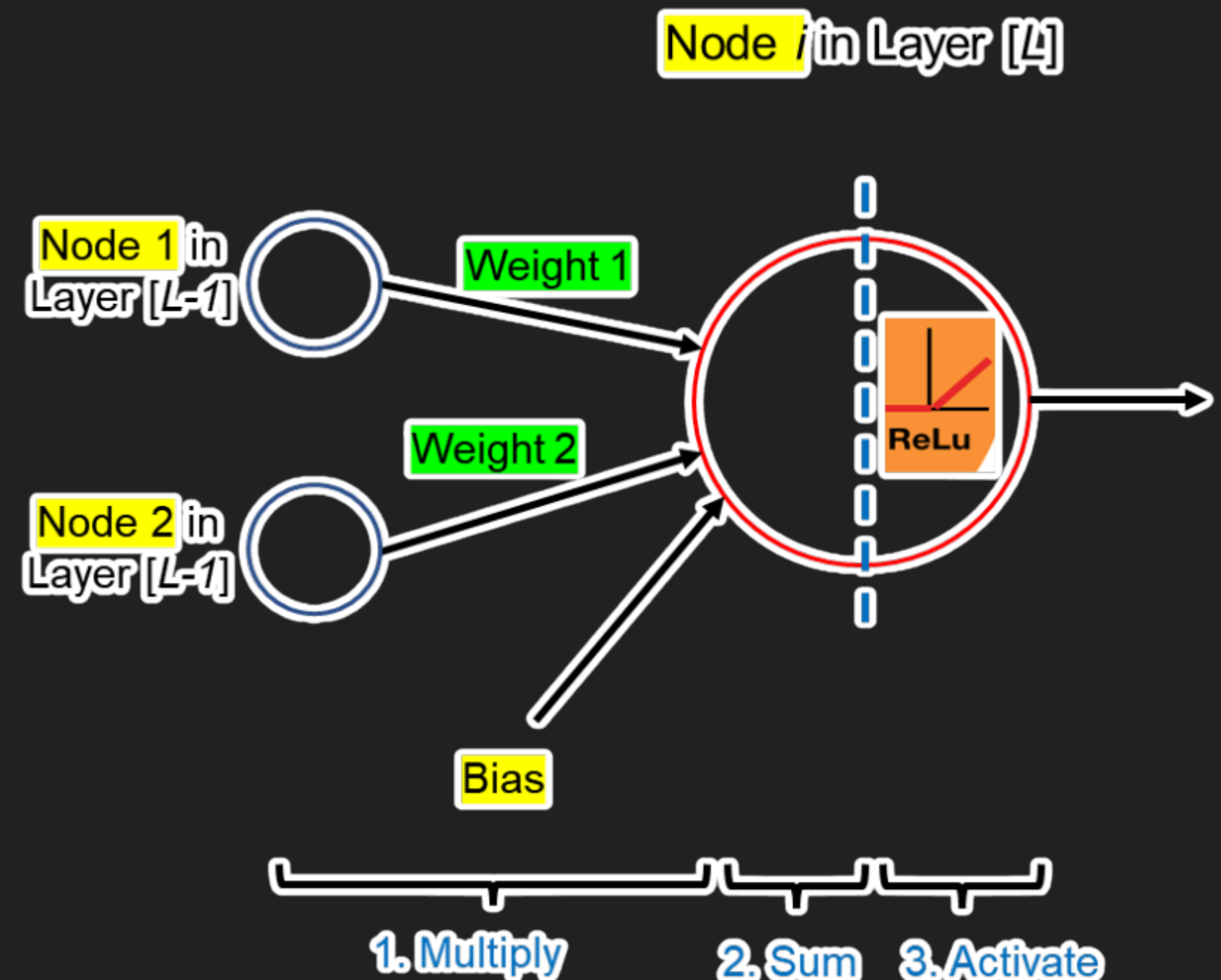
ARTIFICIAL NEURAL NETWORK

- **Most typical form of neural networks model**
- **Each node is fully connected to all nodes of next layer**
- **Final output may be**
 - real-valued output (regression)
 - a set of probabilities (classification)
- **Input Layer:** load and store raw input data
- **Hidden Layer:** contained learned information of the raw training data
- **Output Layer:** get prediction from the network



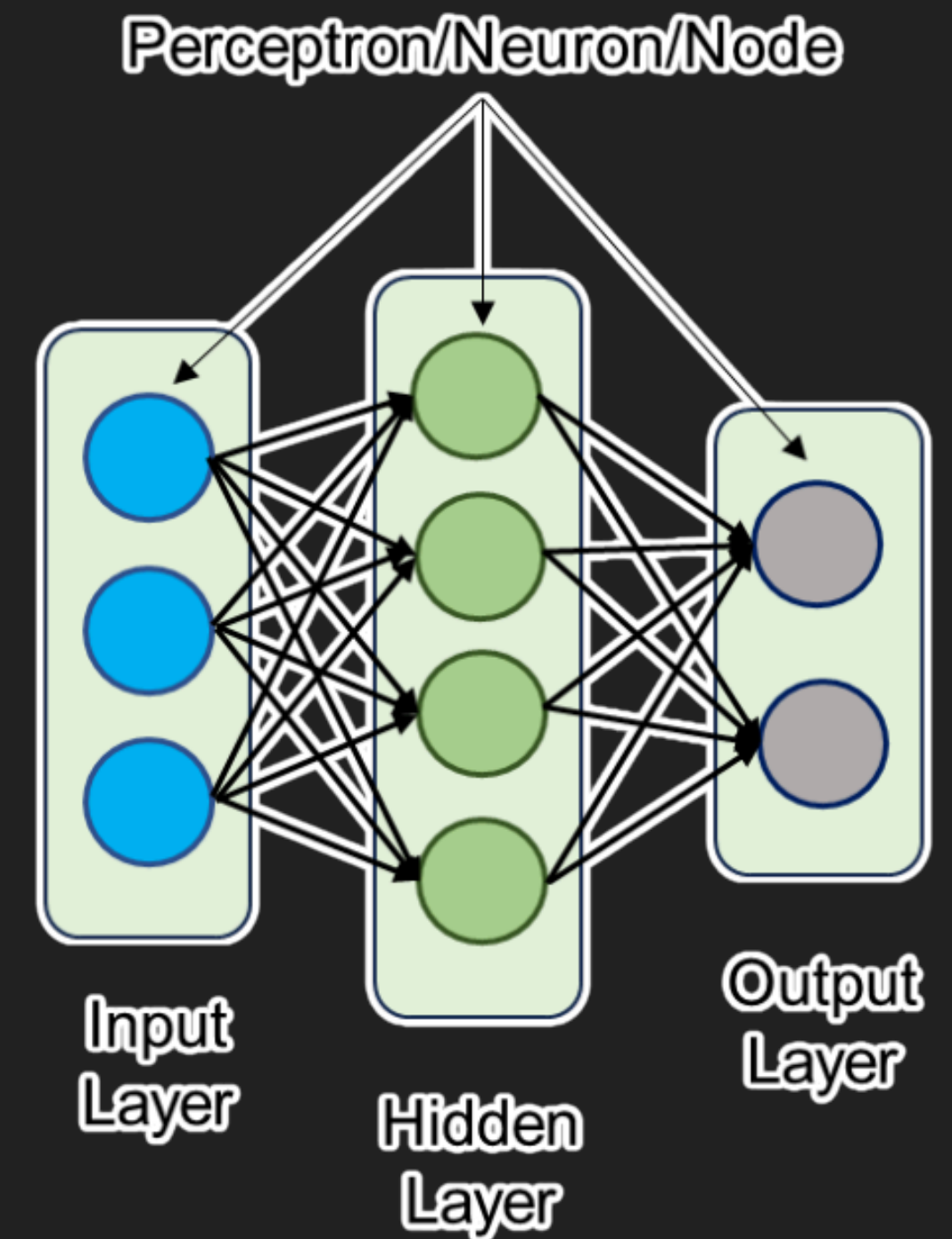
COMPONENTS OF SIMPLE ANN

- **Each neuron has:**
 - Incoming weight and bias value
 - An activation function
- **The activation function (e.g. ReLU)**
 - Transforms the combination of inputs, weights, and biases.
 - The transformed value is the input of next node.
 - Adds non-linearity to the network (allows networks to learn non-linear functions).



PHASES OF ANN: TRAINING

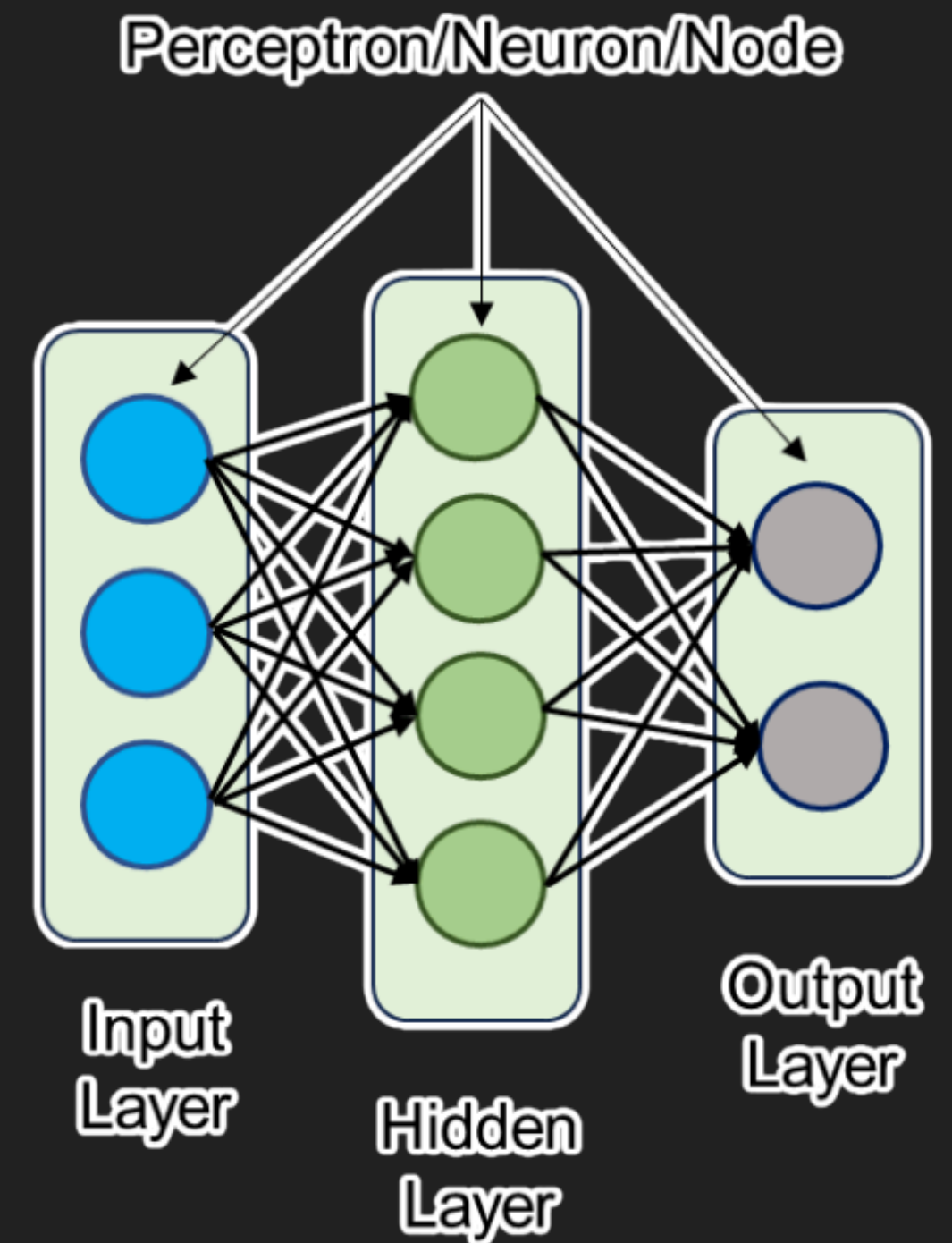
- **Divide dataset into training data and testing data.**
- **Training Steps:**
- **With the training data,**
 - a. Initialize parameters (weights & biases)
 - b. Do forward propagation – Get prediction
 - c. Compute loss (difference between labels & predictions)
 - d. Do backward propagation – compute gradient loss & update parameters (weights & biases)
 - e. Repeat until neural network converges to final weights and biases values.



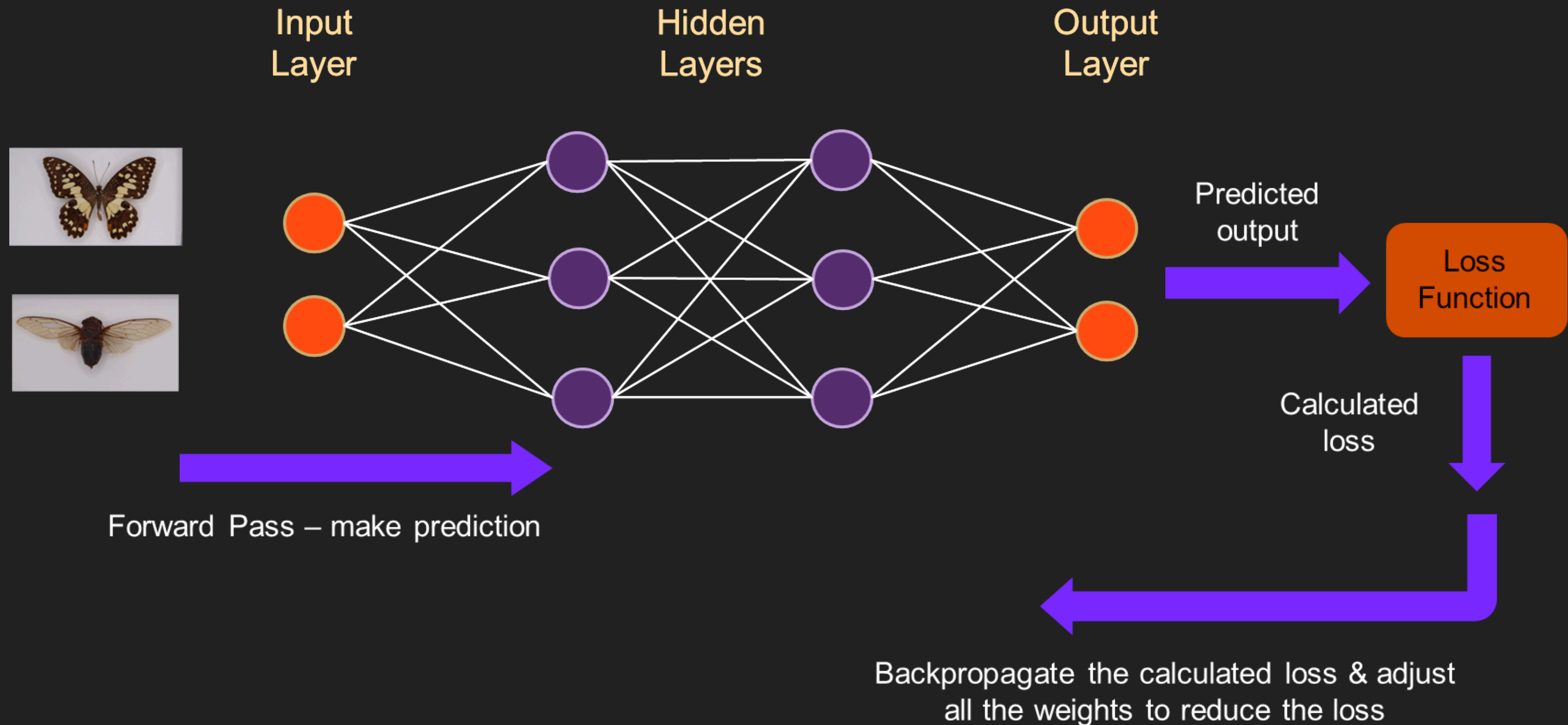
PHASES OF ANN: INFERENCE

In simple terms, inferencing is the process of using a trained AI model to make a prediction based on new, real-world data.

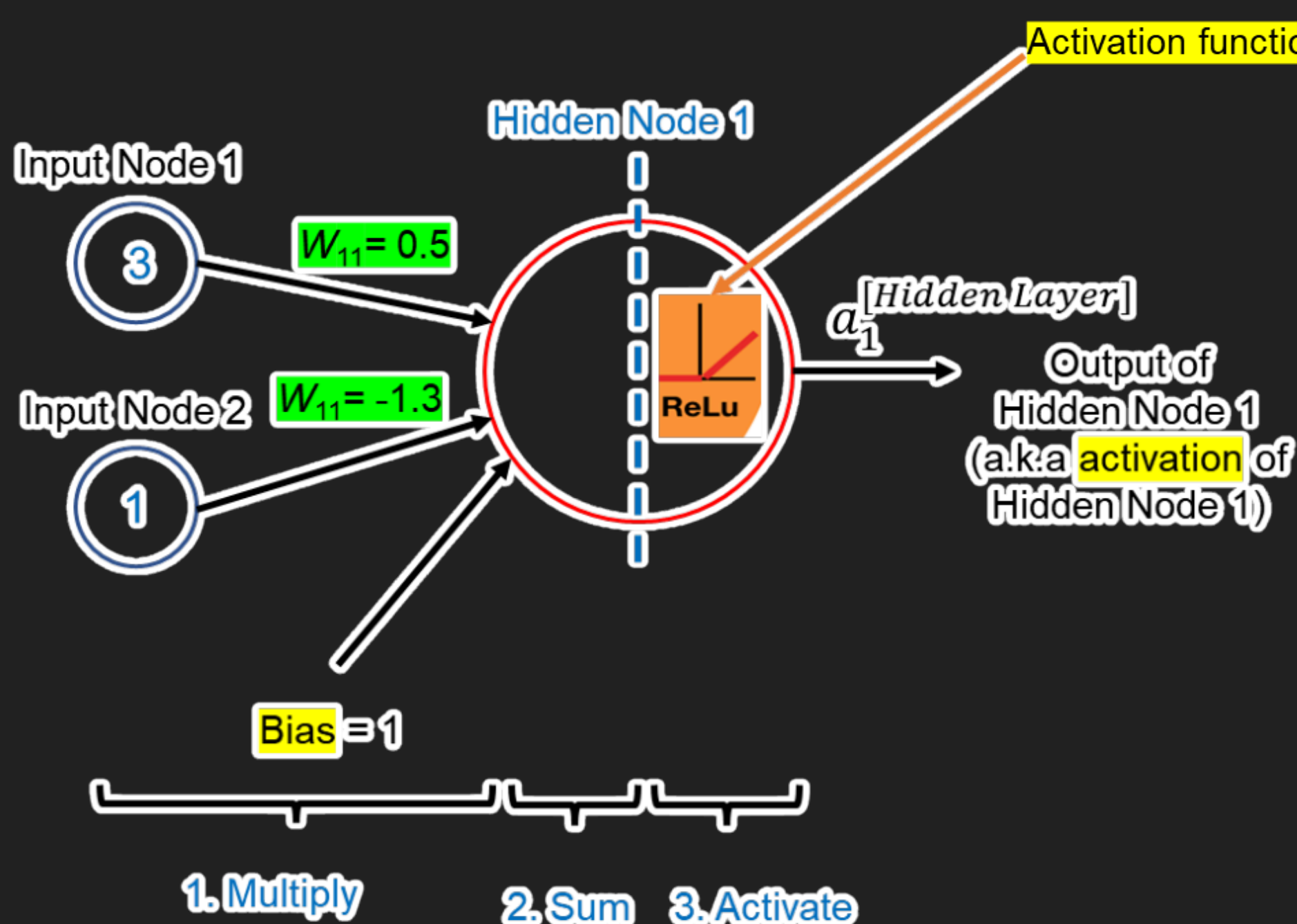
- **Inferencing Steps:**
- **With the testing data and final weights and biases values,**
 - a. Do forward propagation ONLY – Get prediction
 - b. Compute loss (difference between labels & predictions)



GENERAL OPERATION OF ANN



PERCEPTRON/NEURON/NODE

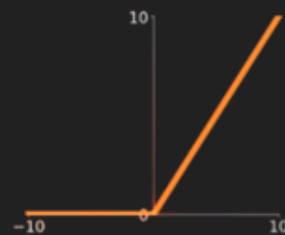


Given the ReLU function (as above) as the activation function of the neuron, calculate the value of $a_1^{[Hidden Layer]}$

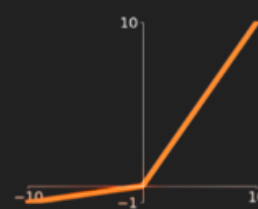
ACTIVATION FUNCTIONS

Hidden Layer Activation Functions

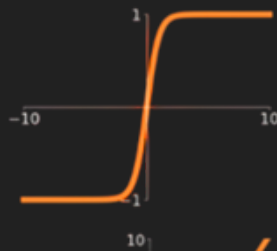
ReLU
 $\max(0, x)$



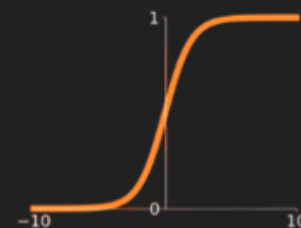
Leaky ReLU
 $\max(0.1x, x)$



tanh
 $\tanh(x)$



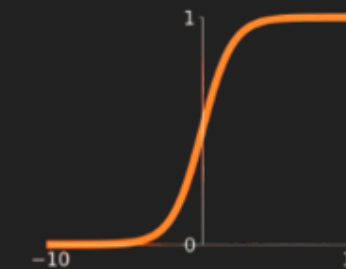
Sigmoid
 $\sigma(x) = \frac{1}{1+e^{-x}}$



Output Layer Activation Functions

Sigmoid

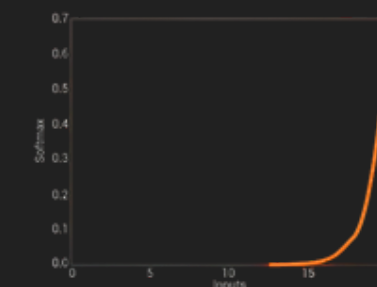
$$\sigma(x) = \frac{1}{1+e^{-x}}$$



–Usually for binary classification problem

Softmax

$$S(f_{y_i}) = \frac{e^{f_{y_i}}}{\sum_j e^{f_j}}$$



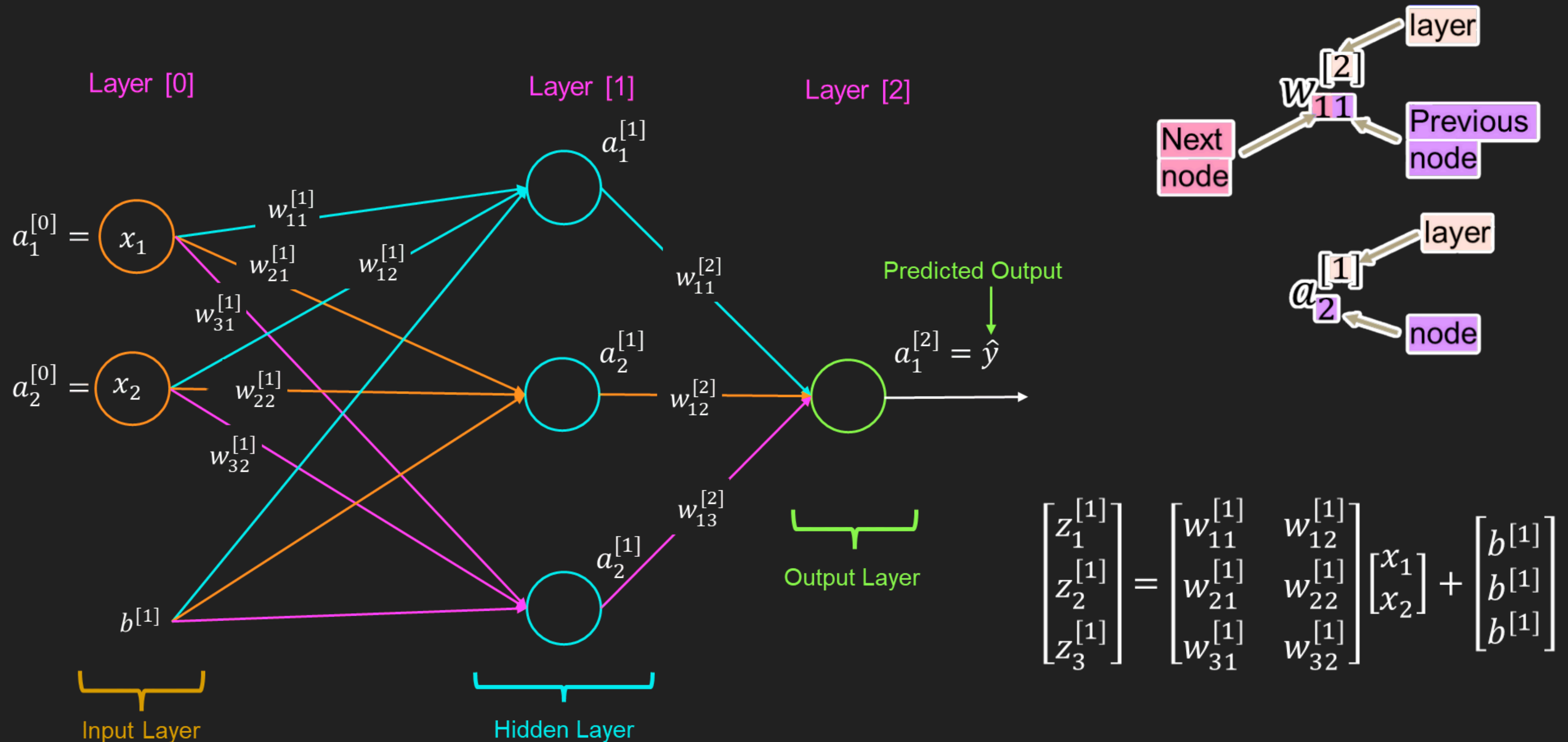
–A generalized version of sigmoid function
–For binary and multiclass classification problem

Linear



–Could be used for regression problem

FRONT PROPAGATION MECHANISM



NUMPY

NumPy (short for Numerical Python) is a fundamental library for the Python programming language.

Its main purpose is to provide a powerful object for multi-dimensional arrays (called an ndarray) and a collection of high-level mathematical functions to operate on these arrays efficiently.

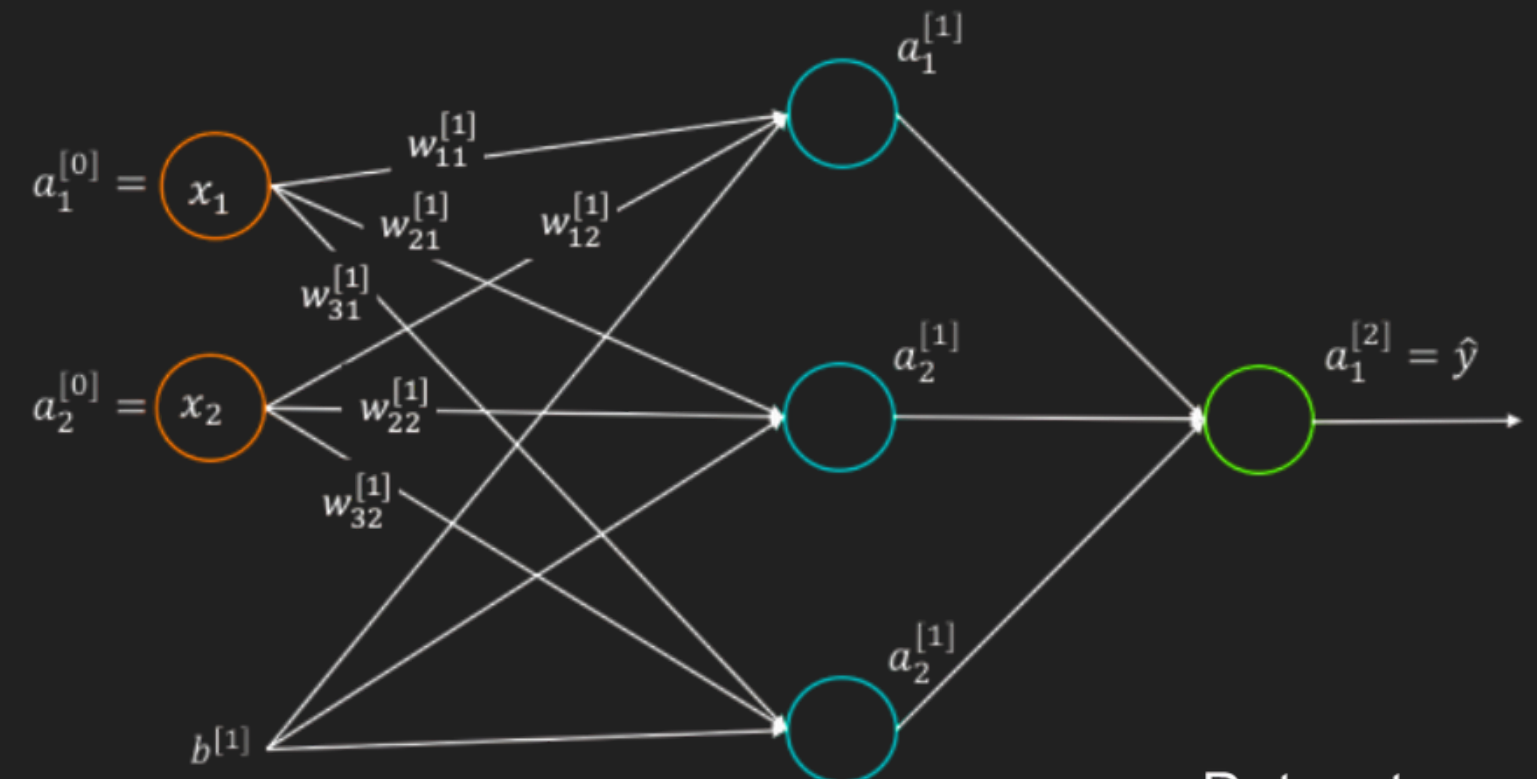
NumPy operations are much faster than using standard Python lists for calculations. This is because its core is written in C and it processes entire arrays at once (a feature called vectorization) rather than one element at a time.



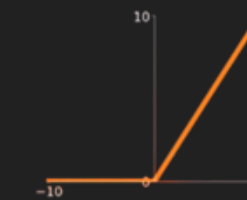
FORWARD PROPAGATION

Copy the Google Colab Notebook to your own Drive and then edit to achieve the following:

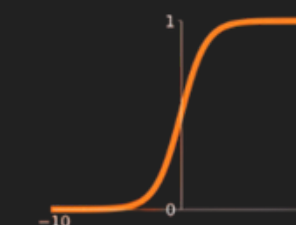
1. Define a ReLU activation function
2. Define a sigmoid activation function
3. Prepare a dataset
4. Define a forward propagation function for a neural network that:
 - a. Has one only one hidden layer (of course there must be an input layer and an output layer)
 - b. Accepts input data and produce predicted output
5. Perform forward propagation for the given ANN architecture and print the activation values for each layer based on the given dataset.



ReLU
 $\max(0, x)$



Sigmoid
 $\sigma(x) = \frac{1}{1+e^{-x}}$



Dataset

i	x_1	y
1	0.5	2
2	-3.2	7

LOSS FUNCTION & COST FUNCTION

i	x_1	x_2	y	$\hat{y}$	$Loss = y - \hat{y}$
1	$x_1^{(1)}$	$x_2^{(1)}$	$y^{(1)}$	$\hat{y}^{(1)}$	$L^{(1)} = y^{(1)} - \hat{y}^{(1)}$
2	$x_1^{(2)}$	$x_2^{(2)}$	$y^{(2)}$	$\hat{y}^{(2)}$	$L^{(2)} = y^{(2)} - \hat{y}^{(2)}$
$\vdots$					
m	$x_1^{(m)}$	$x_2^{(m)}$	$y^{(m)}$	$\hat{y}^{(m)}$	$L^{(m)} = y^{(m)} - \hat{y}^{(m)}$

- **Loss** refers to error for one data sample only. If $y^{(i)} - \hat{y}^{(i)}$ is large, the loss value is high, and vice versa.
- **Cost/Loss function** refers to an average of the loss functions over an entire dataset.

Therefore,

$$\mathbf{Loss} = L^{(i)}(y, \hat{y}) = y^{(i)} - \hat{y}^{(i)}$$

$$\mathbf{Total Loss} = L^{(1)} + L^{(2)} + \dots + L^{(m)}$$

COMMON LOSS FUNCTION

For Regression

Cost Functions:

- (i) Mean Absolute Error (MAE) (L1 loss)
- (ii) Huber Loss
- (iii) **Mean Square Error (MSE) (L2 loss)**

$$J(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2$$

where:

N : number of data points

$y^{(i)}$: actual value (label) for i^{th} data

$\hat{y}^{(i)}$: predicted value for i^{th} data

For Binary Classification

Cost Function:

- (i) Hinge Loss
- (ii) Weighted Cross Entropy Loss
- (iii) **Binary Cross Entropy (BCE)**

$$\begin{aligned} J(y, \hat{y}) \\ &= - \sum_{i=1}^N y^{(i)} \log_e(\hat{y}^{(i)}) + (1 - y^{(i)}) \log_e(1 - \hat{y}^{(i)}) \end{aligned}$$

where:

N : number of data points

$y^{(i)}$: actual value (label) for i^{th} data

$\hat{y}^{(i)}$: predicted value for i^{th} data

COMMON LOSS FUNCTION

For Multiclass Classification

Cost Function: **Cross Entropy (CE)/Logistic Loss Function**

$$s(y^i) = \frac{e^{y_i}}{\sum_j^M e^{y_j}}$$

$$L(W, b) = - \sum_{i=1}^N \sum_{j=1}^M y_{i,j} \times \log S_{i,j}$$

where:

N : number of data points

$y^{(i)}$: actual value (label) for i^{th} data

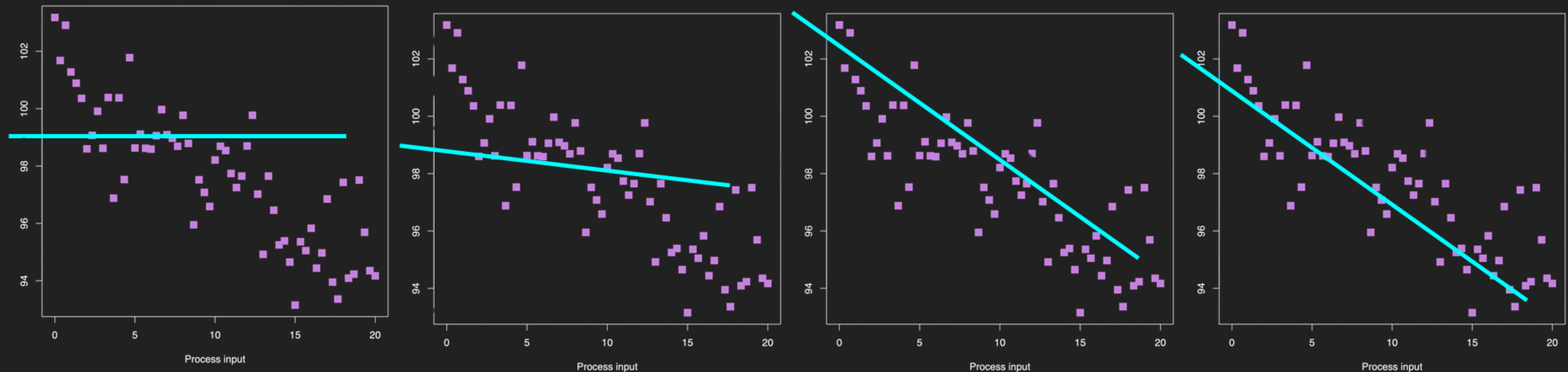
$\hat{y}^{(i)}$: predicted value for i^{th} data

S : softmax function

TRAINING OF NEURAL NETWORK

What is the goal of training?

Adjust the weights and biases to achieve a set with the smallest error, by optimizing (*minimize*) the loss function.



For example in Linear Regression,

$$\hat{y} = Wx + B$$

Goal: Find good values for W and B such that the difference between points and line is minimized

BACK PROPAGATION

We use optimization algorithm such as the **Gradient Descent** algorithm to **automatically find the values of the weights** $\{w_i^{[l]}, b_i^{[l]}\}$, such that the cost (of the error) is minimized.

Gradient Descent Algorithm:

$$w_{new} = w_{old} - \alpha \frac{\partial J}{\partial w}$$
$$b_{new} = b_{old} - \alpha \frac{\partial J}{\partial b}$$

Need these values to update w and b .

We start from the error at the **last layer**.

BACK PROPAGATION

- In backpropagation (BP), we aim to adjust (i.e. optimize) the values of $w^{[1]}$, $b^{[1]}$, $w^{[2]}$, $b^{[2]}$ so that the **prediction error is minimized**.
- We can do this by using optimization algorithm such as the '**Gradient Descent**' algorithm.

- *Gradient Descent is an optimization algorithm to update values of w and b such that J (the cost of error) is minimised .*

$$\begin{aligned} w_{new} &= w_{old} - \alpha \frac{\partial J}{\partial w} \\ b_{new} &= b_{old} - \alpha \frac{\partial J}{\partial b} \end{aligned}$$

Need these values to update w and b .

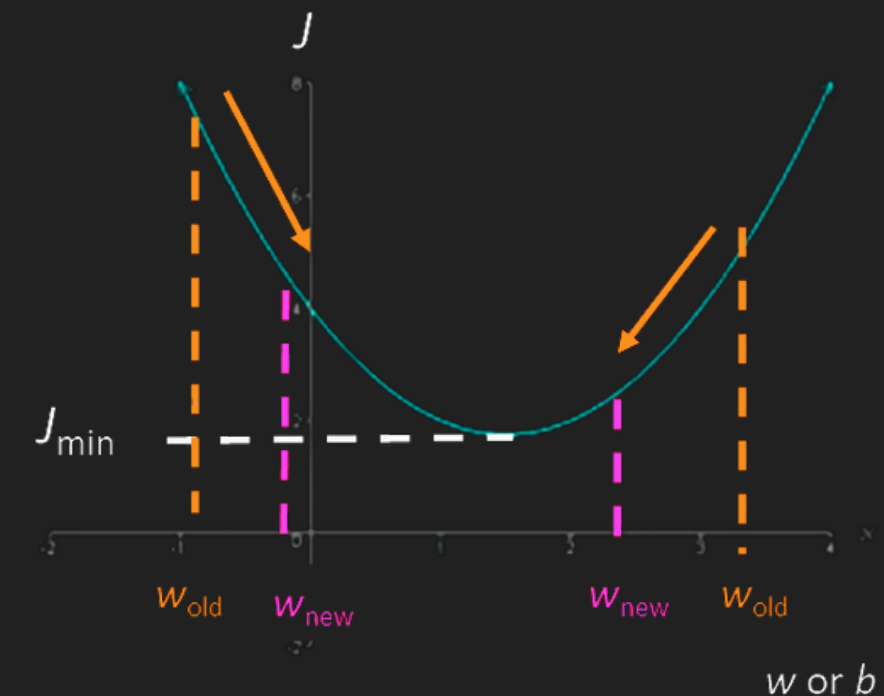
- We therefore have:

Layer 1

$$\begin{aligned} w_{new}^{[1]} &= w_{old}^{[1]} - \alpha \frac{\partial J}{\partial w^{[1]}} dw^{[1]} \\ b_{new}^{[1]} &= b_{old}^{[1]} - \alpha \frac{\partial J}{\partial b^{[1]}} db^{[1]} \end{aligned}$$

Layer 2

$$\begin{aligned} w_{new}^{[2]} &= w_{old}^{[2]} - \alpha \frac{\partial J}{\partial w^{[2]}} dw^{[2]} \\ b_{new}^{[2]} &= b_{old}^{[2]} - \alpha \frac{\partial J}{\partial b^{[2]}} db^{[2]} \end{aligned}$$

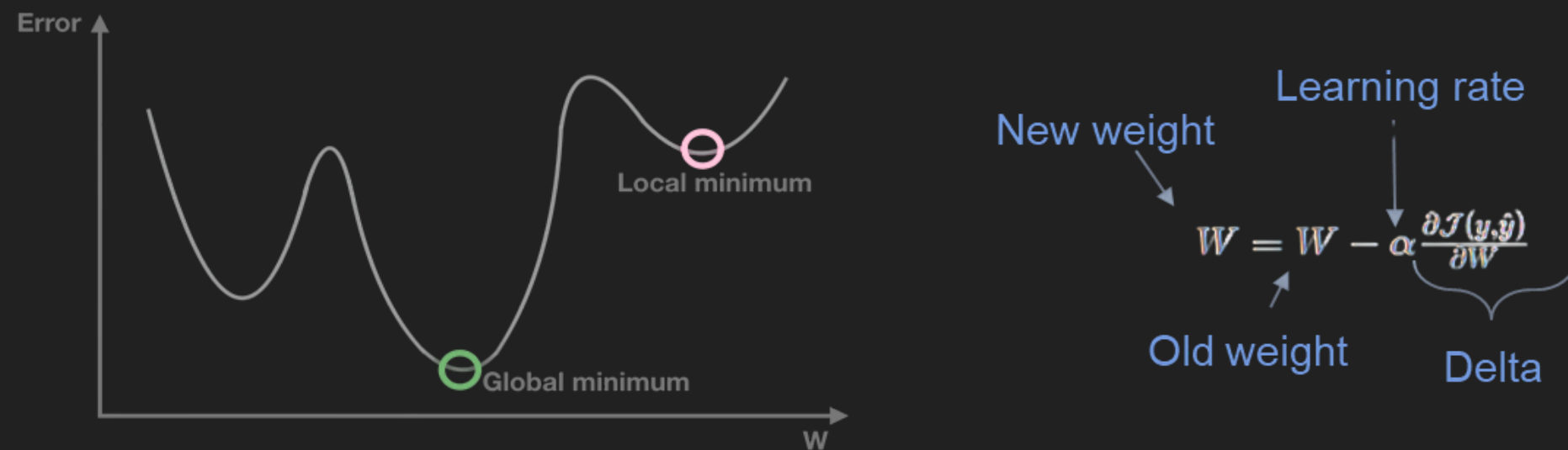


HYPER PARAMETER TUNING

- Configuration settings set before training.
 - Variables which:
 - determine the network structure
 - determine how the network is trained
- **Number of Hidden Layers and Units**
 - **Network Weight Initialization**
 - **Activation Function**
 - **Loss Function & Optimizer/Updater**
 - **Learning Rate**
 - **Batch Training**
 - **Number of Epochs**
 - **Feature Transformation**
 - **Underfit & Overfit Models**
 - **Regularization**
 - **Early Stopping**
 - **Vanishing Gradient**

OPTIMIZER

- In the previous examples, the optimizer we use in our training is the *Gradient Descent* algorithm.
- Training of deep learning algorithms minimize the cost function by going through a series of steps to convergence (an acceptable minimum)

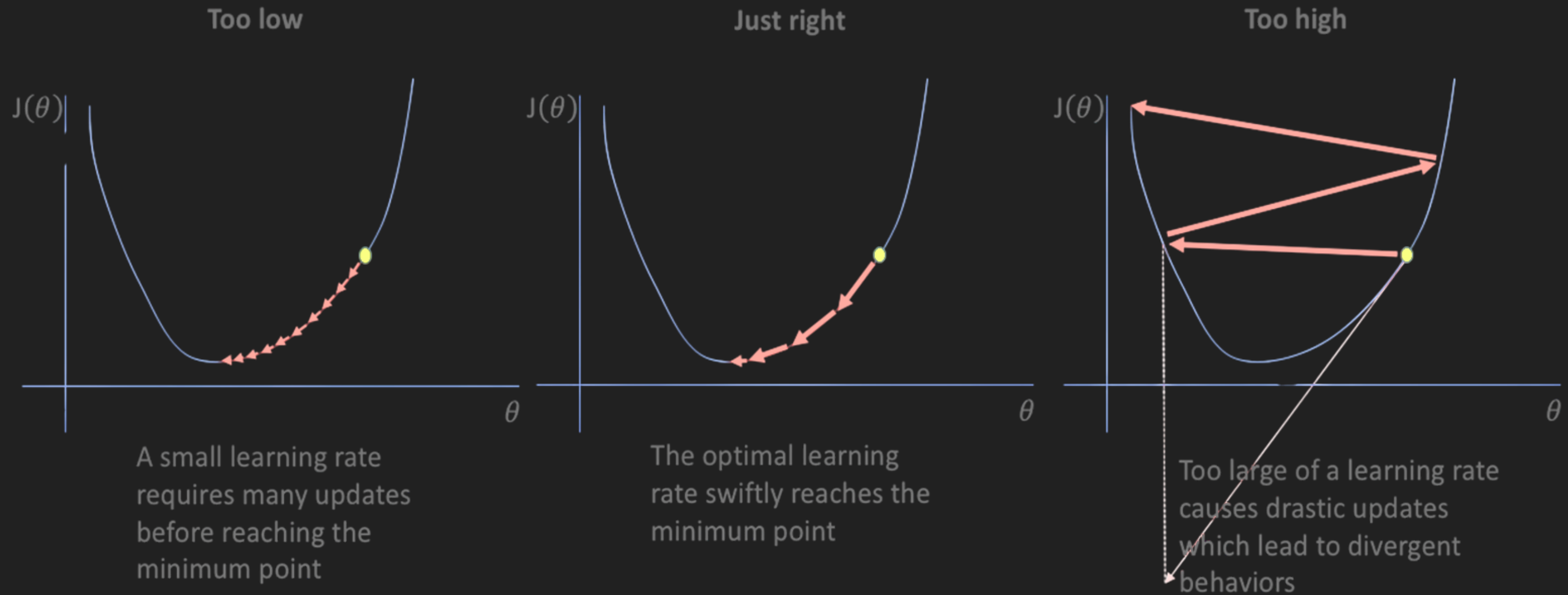


- If $\text{delta} > 0$, increase in weight increases error, then new weight should be smaller.
- If $\text{delta} < 0$, increase in weight decreases error, then new weight should be greater.
- If $\text{delta} = 0$, no more update is needed.

OTHER OPTIMIZER EXAMPLES

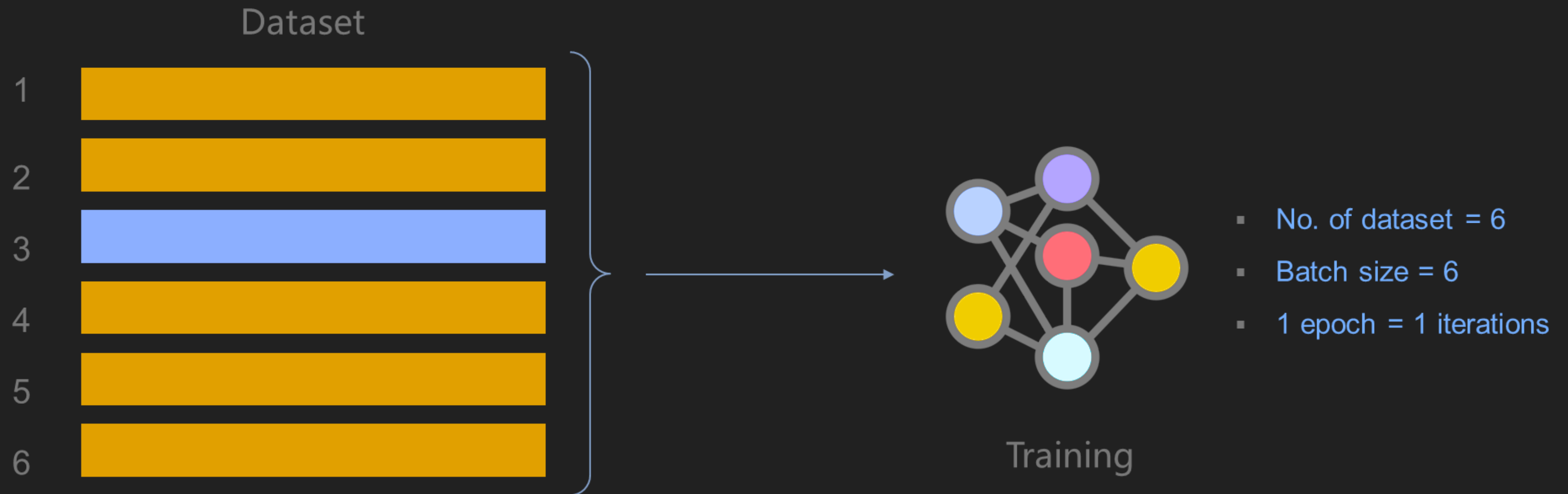
Optimizer	Update rule	Attribute
(Stochastic) Gradient Descent	$W = W - \alpha dW$	- Gradient descent can use parallelization efficiently, but is very slow when the data set is larger the GPU's memory can handle. The parallelization wouldn't be optimal. - Stochastic gradient descent usually converges faster than gradient descent on large datasets, because updates are more frequent. Plus, the stochastic approximation of the gradient is usually precise without using the whole dataset because the data is often redundant. - Of the optimizers profiled here, stochastic gradient descent uses the least memory for a given batch size.
Momentum	$\begin{aligned} V_{dW} &= \beta V_{dW} + (1 - \beta) dW \\ W &= W - \alpha V_{dW} \end{aligned}$	- Momentum usually speeds up the learning with a very minor implementation change. - Momentum uses more memory for a given batch size than stochastic gradient descent but less than RMSprop and Adam.
RMSprop	$\begin{aligned} S_{dW} &= \beta S_{dW} + (1 - \beta) dW^2 \\ W &= W - \alpha \frac{dW}{\sqrt{S_{dW}} + \epsilon} \end{aligned}$	- RMSprop's adaptive learning rate usually prevents the learning rate decay from diminishing too slowly or too fast. - RMSprop maintains per-parameter learning rates. - RMSprop uses more memory for a given batch size than stochastic gradient descent and Momentum, but less than Adam.
<u>Adam</u>	$\begin{aligned} V_{dW} &= \beta_1 V_{dW} + (1 - \beta_1) dW \\ S_{dW} &= \beta_2 S_{dW} + (1 - \beta_2) dW^2 \\ V_{corr_{dW}} &= \frac{V_{dW}}{(1 - \beta_1)^t} \\ S_{corr_{dW}} &= \frac{S_{dW}}{(1 - \beta_2)^t} \\ W &= W - \alpha \frac{V_{corr_{dW}}}{\sqrt{S_{corr_{dW}}} + \epsilon} \end{aligned}$	- The hyperparameters of Adam (learning rate, exponential decay rates for the moment estimates, etc.) are usually set to predefined values (given in the paper), and do not need to be tuned. - Adam performs a form of learning rate annealing with adaptive step-sizes. - Of the optimizers profiled here, Adam uses the most memory for a given batch size. - Adam is often the default optimizer in machine learning.

LEARNING RATE



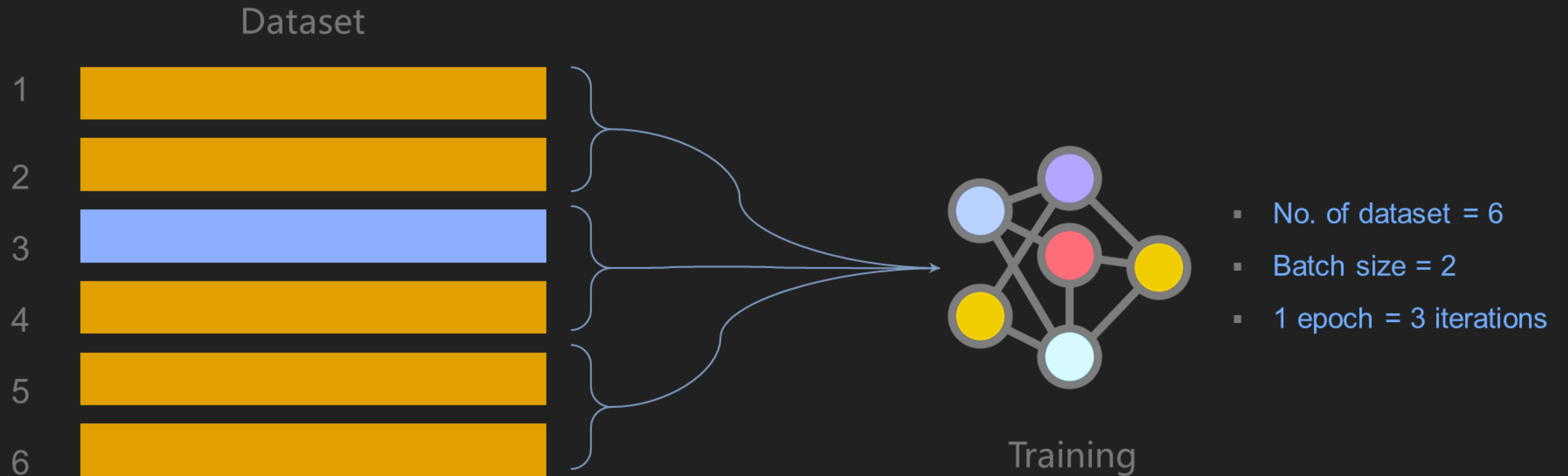
BATCH TRAINING

Use all dataset at once to train the model per iteration



MINI BATCH TRAINING

Use batches of dataset to train the model per iteration



BATCH COMPARISON

	Single Batch Training	Mini Batch Training	Batch Training
Calculate the gradient of the cost function by referring to:	One single sample	A predefined n samples (Mini-batch size)	The whole dataset
Strengths:	• Fastest convergence (less computation) • Easier to escape shallow local minima	• Fast and smooth convergence	• Stable convergence
Drawbacks:	• Zig-zag path to minimum	• Zig-zag path with inappropriate configuration of batch size	• Heavy computation (Increase with total data size)

FEATURE TRANSFORMATION

2 approaches:

- Normalization
 - Accelerate training
 - Regularize model
 - Reduce outlier effect
- Standardization

FEATURE TRANSFORMATION

Min-Max Scaling (min-max normalization)

- scale all values into a fixed range between 0 and 1.
- Faster model convergence and improve accuracy.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Age	Income (RM)	Approve
40	10,000	1
30	5,000	0
25	4,000	1

Min-Max
Normalization
→

Age	Income (RM)	Approve
1.0	1.0	1
0.333	0.167	0
0.0	0.0	1

FEATURE TRANSFORMATION

Standardization (z-score normalization)

- scales the our data to have a mean of 0 and a standard deviation of 1 (unit variance).

$$z = \frac{(x - \mu)}{\sigma}$$

z = standard score
x = observed value
μ = mean of the sample
σ = standard deviation of the sample

Age	Income (RM)	Approve
40	10,000	1
30	5,000	0
25	4,000	1

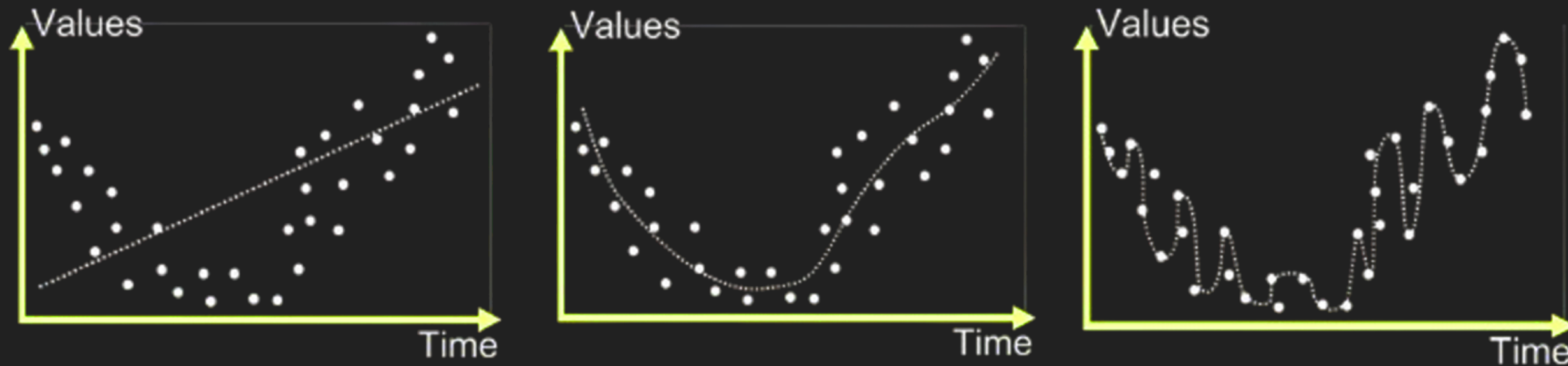
Standardization



Age	Income (RM)	Approve
1.091089	1.140647	1
-0.218218	-0.414781	0
-0.872872	-0.725866	1

OVERFITTING AND UNDERFITTING

- Model Overfitting & Underfitting



- How to prevent underfitting?

1. Increase model complexity
2. Increase features
3. Increase the training epochs or duration of training
4. Decrease or remove the regularization

- How to prevent overfitting?

1. Training on more data
2. Remove irrelevant features
3. Early stopping
4. Regularization

REGULARIZATION

Regularization is a general term for a set of techniques used to prevent overfitting in machine learning models.

The most common types are **L1 (Lasso)** and **L2 (Ridge)** regularization. Early stopping is also considered a form of regularization.

L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

L2 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M W_j^2$$

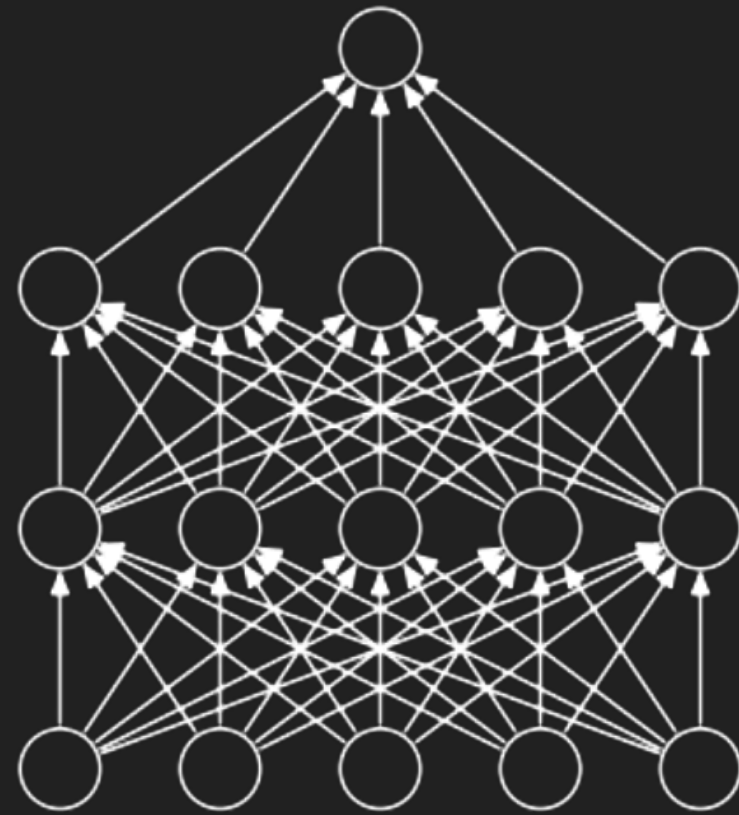
Loss function

Regularization
Term

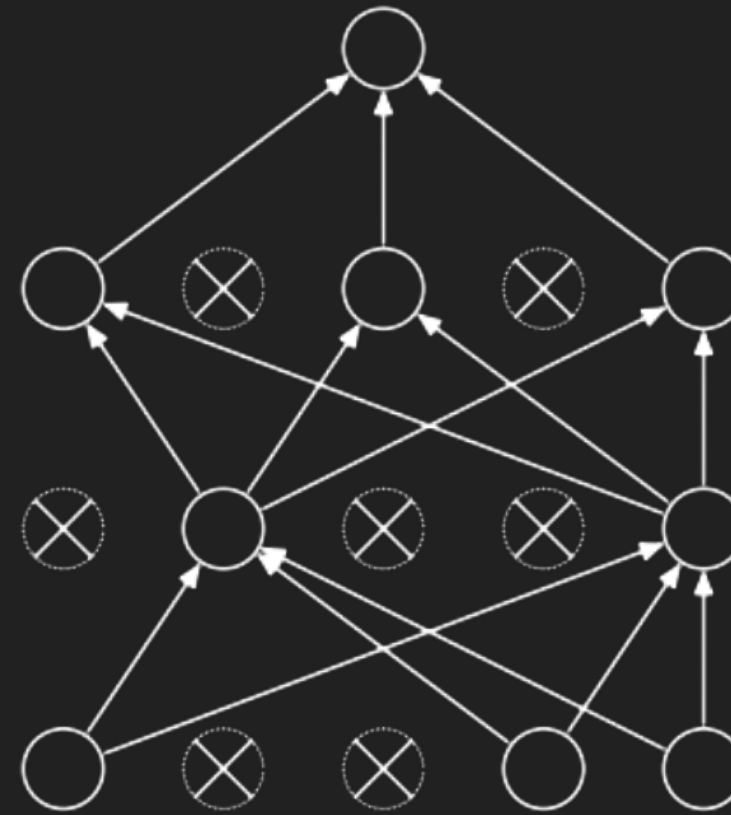
DROP OUT

Dropout is a specific and very popular regularization technique designed for neural networks.

During training, it randomly "drops out" (i.e., deactivates or ignores) a fraction of the neurons in a layer for each training step. This prevents "co-adaptation," where neurons become overly reliant on each other. You can think of it as training many different, smaller neural networks at the same time.



(a) Standard Neural Net



(b) After applying dropout.

COMMON LIBRARIES



PyTorch is an open-source machine learning library primarily developed by Meta AI. It's known for its simplicity, flexibility, and "Python-first" feel.

It's a favorite in the research and academic community because it's excellent for rapid prototyping, complex model development (especially in areas like natural language processing), and easier debugging.



TensorFlow is an open-source machine learning platform developed by Google Brain. It's known for being a powerful, scalable, and production-ready ecosystem.

It's extremely popular in industry and production environments. It offers a robust set of tools for deploying models at scale, including TensorFlow Serving (for servers), TensorFlow Lite (for mobile/IoT devices), and TensorFlow.js (for web browsers).

HANDS-ON SESSION 1

Hands-On Case Study

Regression – Predicting House Price (continuous value)



- **Features (Inputs):** The model will use four features to make its prediction:
 - `square_feet`
 - `num_rooms`
 - `age`
 - `distance_to_city(km)`
- **Target (Output):** The `price` of the house.

HANDS-ON SESSION 2

Hands-On Case Study

Classification – Heart Disease Classification (discrete value)



Numerical Features:	<code>'Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak'</code>
Categorical Features:	<code>'Sex', 'ChestPainType', 'FastingBS', 'RestingECG', 'ExerciseAngina', 'ST_Slope'</code>
Label:	<code>'1' (has heart disease), '0' (no heart disease)</code>

FURTHER RESOURCES

kaggle

LEARN PYTHON, DATA VIZ, PANDAS & MORE |
TUTORIALS | KAGGLE

 PyTorch

WELCOME TO PYTORCH TUTORIALS — PYTORCH
TUTORIALS 2.9.0+CU128 DOCUMENTATION